GRAND VALLEY STATE UNIVERSITY

LITERATURE REVIEW AND RESEARCH GAP

Empirical Analysis Report:

Evolution-Aware Assurance for Agent-Generated Software

What happens after an AI pull request is denied, and what can we learn from it?

October 2026

work in progress…

HOW THE STUDY STARTED

Three papers pointed to one open question

1

Watanabe et al.
TOSEM 2025

Measured how often humans accept agent pull requests.

2

PyQu · Almukhtar et al.
ICSE 2026

A tool that says if a Python change improved quality.

3

Quality & Security Signals
AIware 2026

PyQu, Pylint and Bandit on AI refactoring pull requests.

FIRST QUESTION

What follow-up human action occurs after an AI pull request is denied, and how do developers fix the underlying code?

SECOND QUESTION

When a human corrects an AI change, what reusable obligation does the fix reveal, and can it catch the next mistake?

WHY THIS STUDY

Two things we already know, one thing still unclear

WE ALREADY KNOW · 1

AI coding agents often produce pull requests that later need human correction, during review or after the code is merged.

WE ALREADY KNOW · 2

Software-engineering research has long used past changes and **fixes to learn recurring edit patterns, repair templates and static-analysis rules.**

STILL UNCLEAR

When an AI-generated change is later corrected, can that correction tell us what property the original change should have satisfied?

Not only… "someone changed this code"

But… why the fix was necessary, and which general rule (obligation) was violated

THE STUDY · THREE QUESTIONS

Three simple questions

RQ1 What is corrected?

RQ2 Does it repeat?

RQ3 Can it catch the next one?

Not every correction gives a reusable rule. A major part of the study is to **test whether reusable obligations exist** and, if they do, whether they **add value beyond simpler approaches**:

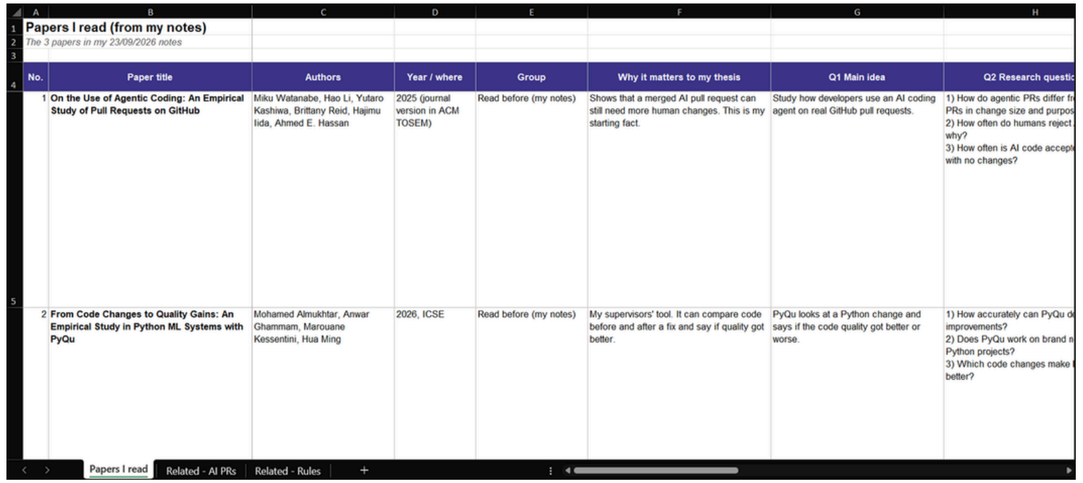
- **Fix-pattern mining**
- **Co-change history**
- **Static-analysis tools**

LITERATURE REVIEW · METHOD

How we found and read the papers

For each paper we answered the same 9 questions

1. Main idea
2. Their research questions
3. Their solution
4. The gap they filled
5. Method and main results
6. Tools
7. Data
8. Future work, what is still open
9. Citation and link



Papers I read (from my notes)

The 3 papers in my 23/09/2026 notes

No.	Paper title	Authors	Year / where	Group	Why it matters to my thesis	Q1 Main idea	Q2 Research questi
1	**On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub**	Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, Ahmed E. Hassan	2025 (journal version in ACM TOSEM)	Read before (my notes)	Shows that a merged AI pull request can still need more human changes. This is my starting fact.	Study how developers use an AI coding agent on real GitHub pull requests.	1) How do agentic PRs differ fr PRs in change size and purpos 2) How often do humans reject why? 3) How often is AI code accepte with no changes?
2	**From Code Changes to Quality Gains: An Empirical Study in Python ML Systems with PyQu**	Mohamed Almukhtar, Anwar Ghammam, Marouane Kessentini, Hua Ming	2026, ICSE	Read before (my notes)	My supervisors' tool. It can compare code before and after a fix and say if quality got better.	PyQu looks at a Python change and says if the code quality got better or worse.	1) How accurately can PyQu d improvements? 2) Does PyQu work on brand n Python projects? 3) Which code changes make better?

Papers I read | Related - AI PRs | Related - Rules

My reading matrix in Excel Sheet

LITERATURE REVIEW · THE MAP

Literature review: Two families

FAMILY A · AI coding agents and their pull requests (24)

A1 · Adoption, data, how agents change code (6)

Watanabe · AIDev · Ogenrwot & Businge · Ottenhof · K. Watanabe · Gong

A2 · Rejection and failure (4)

Ehsani · Alam · Rejection of Fixes · Chouchen (CI)

A3 · Review-time corrections (4)

Khelifi · Haider & Zimmermann · Zhong · Nachuma & Zibran

A4 · After merge: reverts, fixes, maintenance (7)

Oukhay · Khemissi · Who Finishes the Job? · Xia & Miller · Rahman & Shihab · Fang (SATD) · Ouatiti (logging)

A5 · Measuring quality of changes (3)

PyQu · Quality & Security Signals · Cynthia

FAMILY B · Learning rules from code changes and fixes (24)

B1 · Mining history and change patterns (6)

ROSE · DynaMine · Hora · LASE/Sydit · CPatMiner · MELT

B2 · From fixes to patterns and rules (6)

FixMiner · Getafix · HDRepair · Review4Repair · Effendi · Garg & Sengamedu

B3 · An LLM turns a fix into a checker (5)

KNighter · BugStone · BugStone-E2E · TyPatch · SemOpt

B4 · Saying what a change should do (4)

Change Contracts · Le et al. · DeltaSpec · PatchGuru

B5 · Learning from human feedback on AI code (3)

Aggarwal & Ghalaty · Decode (Liang) · Clean-PR (Zhu)

GROUP A · AI AGENTS AND THEIR PULL REQUESTS

Group A: What happens to AI pull requests

Closest and most cited papers

Where Do AI Coding Agents Fail?

Ehsani et al. · MSR 2026

Merged vs not-merged agent PRs on 33,596 PRs; a taxonomy of failures.

71.48%

of agent PRs are merged

Stops at reasons

Why AI Fix PRs Remain Unmerged

Alam, Mondal, Roy · MSR 2026

326 closed AI fix PRs coded by hand: 12 reasons.

22.1%

were “resolved by another PR”

Never reads that PR

Rejection of Agentic Fixes

Abujadallah et al. · 2026

306 rejected AI fixes from AIDev coded by reason.

46.41%

of AI fixes are rejected

Half have no reason

Who Finishes the Job?

Takerngsaksiri et al. · 2026

Pairs merged AI PRs with later fix PRs (human + LLM judge).

27.4%

of the fixes are written by humans

Merged PRs only

Group A shows the problem: AI PRs are often denied, and a human often solves the task, but nobody learns a rule from it.

GROUP A · KEY PAPER · THE 9 REVIEW QUESTIONS

Who Finishes the Job?

Takerngsaksiri, Duong, Barnett · 2026 · arXiv 2609.26847 · follow-up fixes of AI pull requests

Q1 · Main idea
Study the follow-up fixes of merged agent PRs and who writes them.

Q2 · Their research questions
Do agent PRs get more fixes than human PRs? Who writes the fixes?

Q3 · Their solution
They find candidate fixes, then check them with human validation and an LLM judge.

Q4 · Gap they filled
Earlier work did not mine the follow-up fixes of merged agent PRs.

Q5 · Method and results
Fix odds **1.62×** human PRs. In 263 pairs: 69.6% same agent, **27.4% humans**, 3.0% another agent.

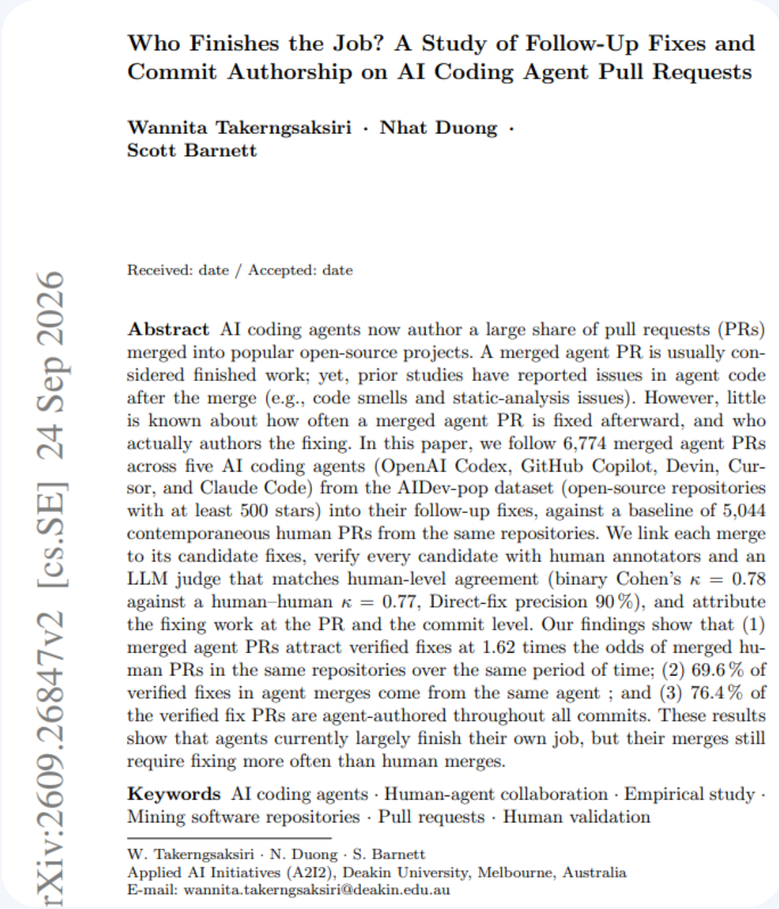
Q6 · Tools
Human validation + LLM judge (kappa 0.78, precision 90%).

Q7 · Data
6,774 merged agent PRs in 891 repos (AIDev-pop). github.com/wannita901/pr-fix-authorship

Q8 · Future work / still open
Merge-time signals barely predict a fix: “the signal may live in the code”.

Q9 · Citation
arxiv.org/html/2609.26847

Difference from my study: They find the pairs, but only for merged PRs, and they do not write rules from them. I do it for denied PRs.



Who Finishes the Job? A Study of Follow-Up Fixes and Commit Authorship on AI Coding Agent Pull Requests

Wannita Takerngsaksiri · Nhat Duong · Scott Barnett

rXiv:2609.26847v2 [cs.SE] 24 Sep 2026

Received: date / Accepted: date

Abstract AI coding agents now author a large share of pull requests (PRs) merged into popular open-source projects. A merged agent PR is usually considered finished work; yet, prior studies have reported issues in agent code after the merge (e.g., code smells and static-analysis issues). However, little is known about how often a merged agent PR is fixed afterward, and who actually authors the fixing. In this paper, we follow 6,774 merged agent PRs across five AI coding agents (OpenAI Codex, GitHub Copilot, Devin, Cursor, and Claude Code) from the AIDev-pop dataset (open-source repositories with at least 500 stars) into their follow-up fixes, against a baseline of 5,044 contemporaneous human PRs from the same repositories. We link each merge to its candidate fixes, verify every candidate with human annotators and an LLM judge that matches human-level agreement (binary Cohen's $\kappa = 0.78$ against a human–human $\kappa = 0.77$, Direct-fix precision 90%), and attribute the fixing work at the PR and the commit level. Our findings show that (1) merged agent PRs attract verified fixes at 1.62 times the odds of merged human PRs in the same repositories over the same period of time; (2) 69.6% of verified fixes in agent merges come from the same agent ; and (3) 76.4% of the verified fix PRs are agent-authored throughout all commits. These results show that agents currently largely finish their own job, but their merges still require fixing more often than human merges.

Keywords AI coding agents · Human-agent collaboration · Empirical study · Mining software repositories · Pull requests · Human validation

W. Takerngsaksiri · N. Duong · S. Barnett
Applied AI Initiatives (A2I2), Deakin University, Melbourne, Australia
E-mail: wannita.takerngsaksiri@deakin.edu.au

GROUP A · KEY PAPER · THE 9 REVIEW QUESTIONS

AIDev: Studying AI Coding Agents on GitHub

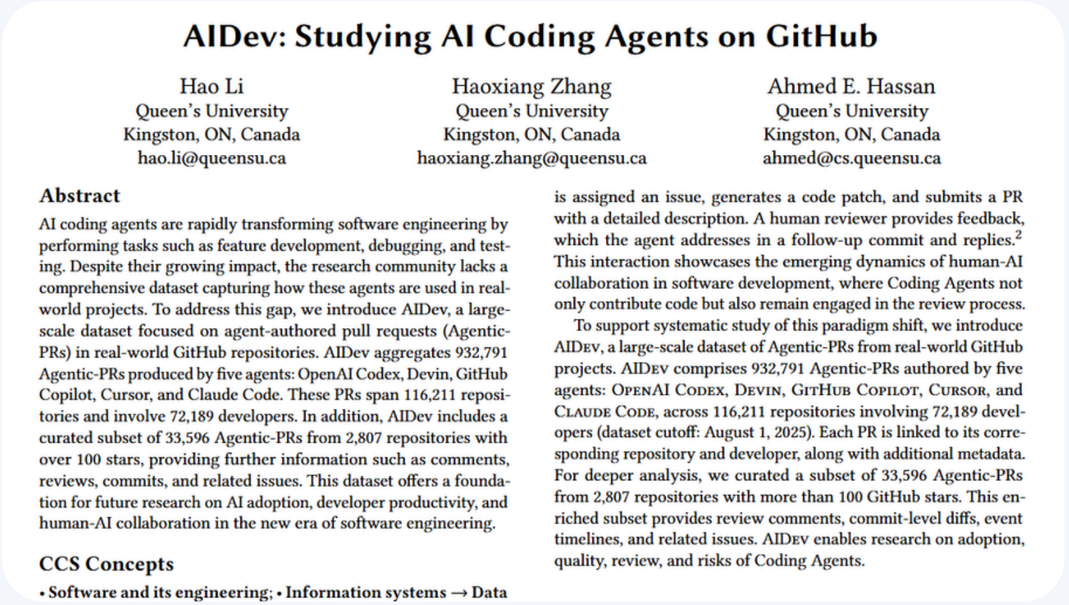
Hao Li, Haoxiang Zhang, Ahmed E. Hassan · MSR 2026

Q1 · Main idea
A public dataset of pull requests written by coding agents.

Q2 · Their research questions
How are coding agents used on GitHub, and what do their PRs look like?

Q3 · Their solution
They collect agent PRs and publish a smaller curated set with reviews, commits and issues.

Q4 · Gap they filled
Researchers had no shared, large set of real agent PRs.



AIDev: Studying AI Coding Agents on GitHub

Hao Li
Queen's University
Kingston, ON, Canada
hao.li@queensu.ca

Haoxiang Zhang
Queen's University
Kingston, ON, Canada
haoxiang.zhang@queensu.ca

Ahmed E. Hassan
Queen's University
Kingston, ON, Canada
ahmed@cs.queensu.ca

Abstract

AI coding agents are rapidly transforming software engineering by performing tasks such as feature development, debugging, and testing. Despite their growing impact, the research community lacks a comprehensive dataset capturing how these agents are used in real-world projects. To address this gap, we introduce AIDev, a large-scale dataset focused on agent-authored pull requests (Agentic-PRs) in real-world GitHub repositories. AIDev aggregates 932,791 Agentic-PRs produced by five agents: OpenAI Codex, Devin, GitHub Copilot, Cursor, and Claude Code. These PRs span 116,211 repositories and involve 72,189 developers. In addition, AIDev includes a curated subset of 33,596 Agentic-PRs from 2,807 repositories with over 100 stars, providing further information such as comments, reviews, commits, and related issues. This dataset offers a foundation for future research on AI adoption, developer productivity, and human-AI collaboration in the new era of software engineering.

CCS Concepts

• Software and its engineering; • Information systems → Data

is assigned an issue, generates a code patch, and submits a PR with a detailed description. A human reviewer provides feedback, which the agent addresses in a follow-up commit and replies.[2] This interaction showcases the emerging dynamics of human-AI collaboration in software development, where Coding Agents not only contribute code but also remain engaged in the review process.

To support systematic study of this paradigm shift, we introduce AIDev, a large-scale dataset of Agentic-PRs from real-world GitHub projects. AIDev comprises 932,791 Agentic-PRs authored by five agents: OpenAI Codex, Devin, GitHub Copilot, Cursor, and Claude Code, across 116,211 repositories involving 72,189 developers (dataset cutoff: August 1, 2025). Each PR is linked to its corresponding repository and developer, along with additional metadata. For deeper analysis, we curated a subset of 33,596 Agentic-PRs from 2,807 repositories with more than 100 GitHub stars. This enriched subset provides review comments, commit-level diffs, event timelines, and related issues. AIDev enables research on adoption, quality, review, and risks of Coding Agents.

Q5 · Method and results
932,791 PRs from Codex, Devin, Copilot, Cursor and Claude Code in 116,211 repos. Curated: **33,596 PRs** in 2,807 repos (100+ stars).

Q6 · Tools
GitHub APIs and a data-building pipeline.

Q7 · Data
huggingface.co/datasets/hao-li/AIDev (live page now v4: 2,743,854 PRs, six agents).

Q8 · Future work / still open
The dataset does not recover rules by itself. I must pick one version and cite it.

Q9 · Citation
arxiv.org/abs/2602.09185

Difference from my study: It gives me the raw material: the denied AI PRs. I turn the pairs from it into rules.

GROUP B · RULES AND CHECKERS FROM CODE CHANGES

Group B: turning fixes into rules and checkers

Closest and most cited papers

Mining Static Analysis Rules

Effendi et al. · ICSE-SEIP 2023

Groups repeated human bug fixes and turns them into rules in CodeGuru.

73%

of rule warnings accepted

Human fixes only

KNighter

Yang et al. · SOSP 2025

An LLM writes a checker from one Linux patch, then scans the kernel.

92

new Linux bugs found

Human fixes only

CPatMiner + Python follow-up

ICSE 2019 · ICSE 2022

Find code changes that developers repeat across projects.

58–75%

of patterns repeat across projects or developers

Human changes only

Self-Improving AI Agents

Aggarwal & Ghalaty · 2026 (Microsoft)

Each reviewer comment becomes a written rule the agent rereads.

0

repeats of 9 tracked mistakes

Plain text, one company

Group B shows the method: fixes can become useful rules and they repeat, but only learned from human code (or plain text).

GROUP B · KEY PAPER · THE 9 REVIEW QUESTIONS

Self-Improving AI Coding Agents

Aditya Aggarwal, Nahid Farhady Ghalaty · 2026 · Microsoft · the closest paper on rules for agents

Q1 · Main idea
A human review comment becomes a saved rule. The agent reads the rule before sending code again.

Q2 · Their research questions
Can an agent stop repeating the same mistake if it keeps the review comments, without retraining?

Q3 · Their solution
Store rules in a file, turn them into a checklist, and run the checklist before submitting.

Q4 · Gap they filled
Agents forgot corrections between sessions.

Q5 · Method and results
In one company, rules grew from **5 to 18**. In the measured sessions, those mistakes did not come back.

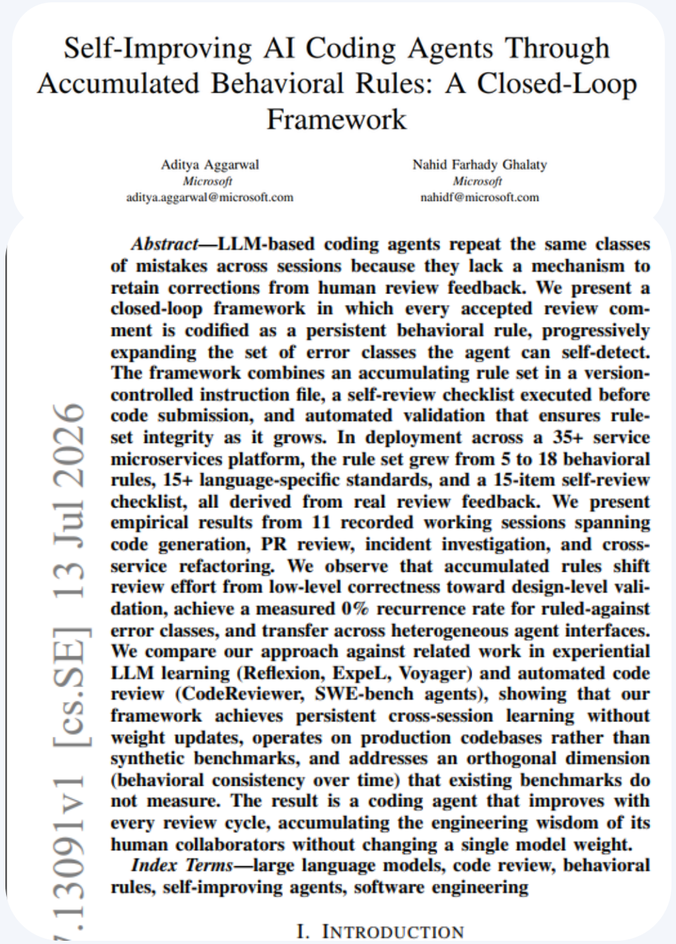
Q6 · Tools
A version-controlled instruction file, a self-review checklist, checks that the rule file stays valid.

Q7 · Data
A real system with 35+ services and 11 recorded sessions. No public dataset.

Q8 · Future work / still open
Tested in one company only. Not shown on new AI PRs across many projects.

Q9 · Citation
arxiv.org/abs/2607.13091

Difference from my study: Source of their rule: the reviewer's sentence. Mine: the code of the human fix. They test in one company; I test across projects.



Self-Improving AI Coding Agents Through Accumulated Behavioral Rules: A Closed-Loop Framework

Aditya Aggarwal
Microsoft
aditya.aggarwal@microsoft.com

Nahid Farhady Ghalaty
Microsoft
nahidf@microsoft.com

***Abstract*—LLM-based coding agents repeat the same classes of mistakes across sessions because they lack a mechanism to retain corrections from human review feedback. We present a closed-loop framework in which every accepted review comment is codified as a persistent behavioral rule, progressively expanding the set of error classes the agent can self-detect. The framework combines an accumulating rule set in a version-controlled instruction file, a self-review checklist executed before code submission, and automated validation that ensures rule-set integrity as it grows. In deployment across a 35+ service microservices platform, the rule set grew from 5 to 18 behavioral rules, 15+ language-specific standards, and a 15-item self-review checklist, all derived from real review feedback. We present empirical results from 11 recorded working sessions spanning code generation, PR review, incident investigation, and cross-service refactoring. We observe that accumulated rules shift review effort from low-level correctness toward design-level validation, achieve a measured 0% recurrence rate for ruled-against error classes, and transfer across heterogeneous agent interfaces. We compare our approach against related work in experiential LLM learning (Reflexion, ExpeL, Voyager) and automated code review (CodeReviewer, SWE-bench agents), showing that our framework achieves persistent cross-session learning without weight updates, operates on production codebases rather than synthetic benchmarks, and addresses an orthogonal dimension (behavioral consistency over time) that existing benchmarks do not measure. The result is a coding agent that improves with every review cycle, accumulating the engineering wisdom of its human collaborators without changing a single model weight.*

***Index Terms*—large language models, code review, behavioral rules, self-improving agents, software engineering**

.13091v1 [cs.SE] 13 Jul 2026

I. INTRODUCTION

GROUP B · KEY PAPER · THE 9 REVIEW QUESTIONS

Mining Static Analysis Rules from Code Changes

Effendi, Cirisci, Mukherjee, Nguyen, Tripp · ICSE-SEIP 2023 · Amazon ·

Q1 · Main idea
Repeated human bug fixes can become static-analysis rules that work in more than one language.

Q2 · Their research questions
Can we mine real bug-fix commits, group the ones that mean the same, and ship them as review rules?

Q3 · Their solution
Represent code as a language-independent graph (MU), cluster similar changes, and turn clusters into rules.

Q4 · Gap they filled
Writing rules by hand for many libraries and languages does not scale.

Q5 · Method and results
From fewer than 600 clusters: **62 rules** for Java, JavaScript and Python. In CodeGuru, developers accepted **73%**.

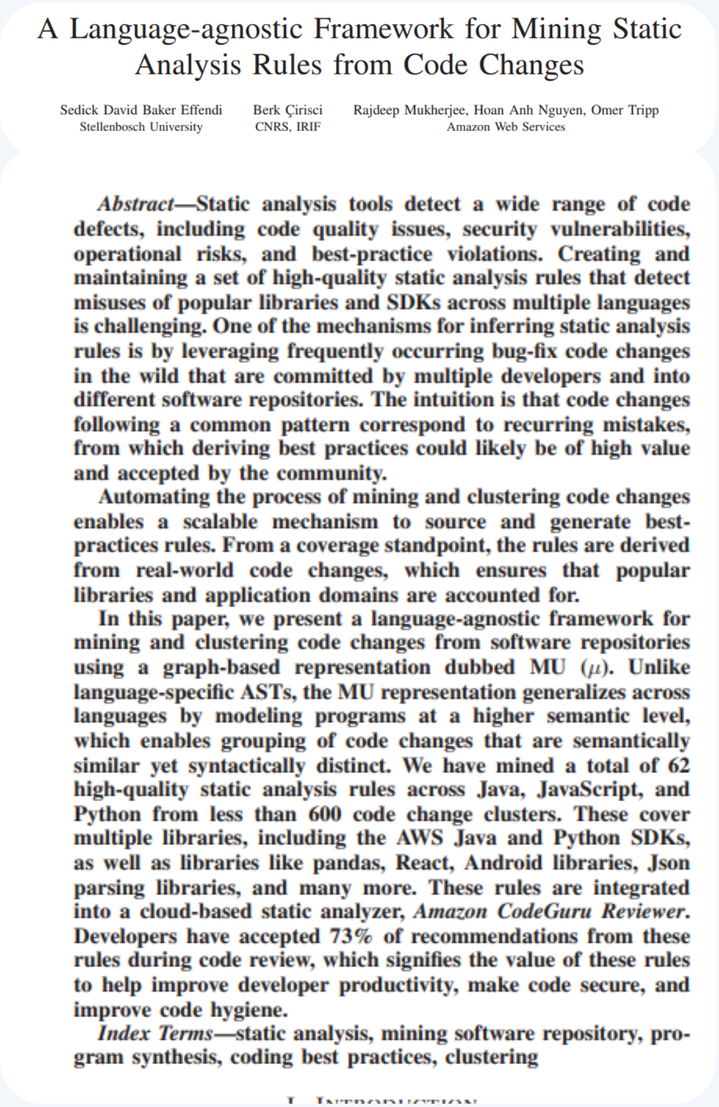
Q6 · Tools
MU graph, change clustering, Amazon CodeGuru Reviewer.

Q7 · Data
Bug-fix commits from real repositories (AWS SDKs, pandas, React, Android).

Q8 · Future work / still open
Input is a normal human bug fix, not an AI change with its later fix.

Q9 · Citation
doi.org/10.1109/ICSE-SEIP58684.2023.00035

Difference from my study: Source: many normal human bug fixes. Mine: an AI change + its later human fix, and we explain why the fix was needed.



A Language-agnostic Framework for Mining Static Analysis Rules from Code Changes

Sedick David Baker Effendi
Stellenbosch University

Berk Çirisci
CNRS, IRIF

Rajdeep Mukherjee, Hoan Anh Nguyen, Omer Tripp
Amazon Web Services

***Abstract*—Static analysis tools detect a wide range of code defects, including code quality issues, security vulnerabilities, operational risks, and best-practice violations. Creating and maintaining a set of high-quality static analysis rules that detect misuses of popular libraries and SDKs across multiple languages is challenging. One of the mechanisms for inferring static analysis rules is by leveraging frequently occurring bug-fix code changes in the wild that are committed by multiple developers and into different software repositories. The intuition is that code changes following a common pattern correspond to recurring mistakes, from which deriving best practices could likely be of high value and accepted by the community.**

Automating the process of mining and clustering code changes enables a scalable mechanism to source and generate best-practices rules. From a coverage standpoint, the rules are derived from real-world code changes, which ensures that popular libraries and application domains are accounted for.

In this paper, we present a language-agnostic framework for mining and clustering code changes from software repositories using a graph-based representation dubbed MU (μ). Unlike language-specific ASTs, the MU representation generalizes across languages by modeling programs at a higher semantic level, which enables grouping of code changes that are semantically similar yet syntactically distinct. We have mined a total of 62 high-quality static analysis rules across Java, JavaScript, and Python from less than 600 code change clusters. These cover multiple libraries, including the AWS Java and Python SDKs, as well as libraries like pandas, React, Android libraries, Json parsing libraries, and many more. These rules are integrated into a cloud-based static analyzer, *Amazon CodeGuru Reviewer*. Developers have accepted 73% of recommendations from these rules during code review, which signifies the value of these rules to help improve developer productivity, make code secure, and improve code hygiene.

***Index Terms*—static analysis, mining software repository, program synthesis, coding best practices, clustering**

I. INTRODUCTION

GROUP B · KEY PAPER · THE 9 REVIEW QUESTIONS

CPatMiner: Semantic Code Change Patterns

H. A. Nguyen, T. N. Nguyen, Dig, S. Nguyen, Tran, Hilton · ICSE 2019 · do changes repeat?

Q1 · Main idea
Developers repeat the same changes. A graph of each change finds repeats that mean the same thing.

Q2 · Their research questions
Do the patterns make sense to developers? Is a graph better than syntax? Do developers share patterns?

Q3 · Their solution
CPatMiner joins before/after dependency graphs into change graphs and mines sub-graphs that repeat.

Q4 · Gap they filled
Syntax-only tools missed changes that look different but mean the same thing.

Q5 · Method and results
70% of 108 surveyed developers: meaningful. **2.1×** more patterns than the baseline. **75%** shared across developers; 35% are bug fixes.

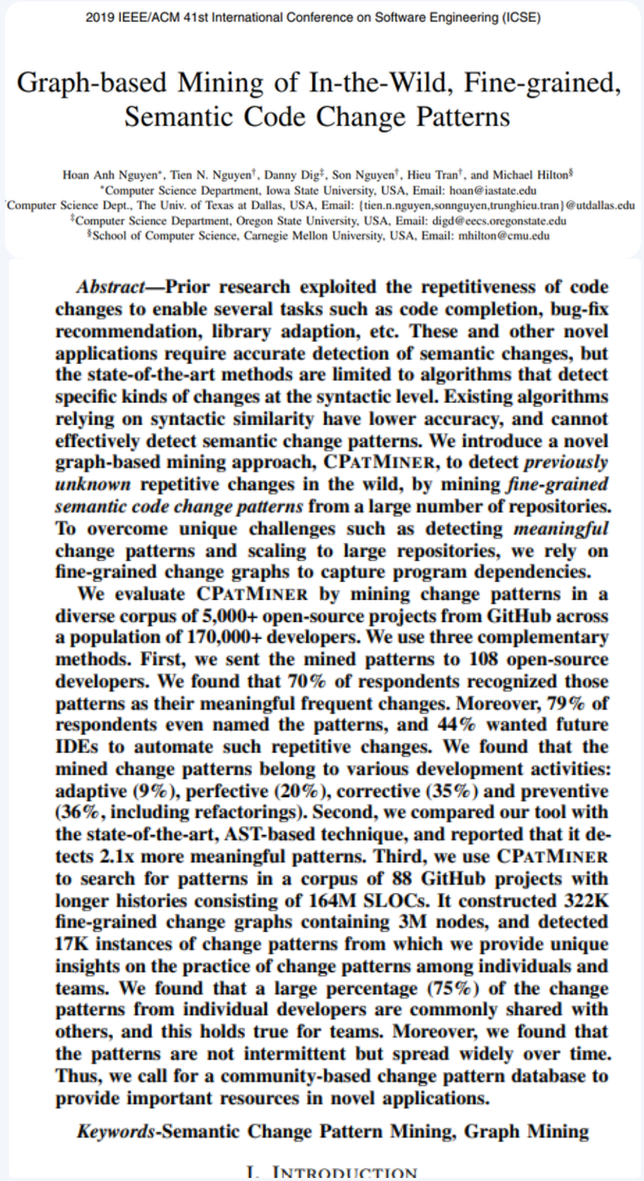
Q6 · Tools
Fine-grained program dependence graphs and graph mining.

Q7 · Data
5,832 Java GitHub projects, 1.1 million change graphs.
nguyenhoan.github.io/CPatMiner

Q8 · Future work / still open
A catalog of patterns, not checkers. 44% of developers want IDEs to automate them.

Q9 · Citation
ieeexplore.ieee.org/document/8812071

Difference from my study: They mine ordinary human changes in Java and stop at a catalog. we mine the human solutions of denied AI PRs and test if they repeat (RQ2).



2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)

Graph-based Mining of In-the-Wild, Fine-grained, Semantic Code Change Patterns

Hoan Anh Nguyen*, Tien N. Nguyen†, Danny Dig‡, Son Nguyen†, Hieu Tran†, and Michael Hilton§
*Computer Science Department, Iowa State University, USA, Email: hoan@iastate.edu
Computer Science Dept., The Univ. of Texas at Dallas, USA, Email: {tien.n.nguyen,sonnguyen,trunghieu.tran}@utdallas.edu
‡Computer Science Department, Oregon State University, USA, Email: digd@eecs.oregonstate.edu
§School of Computer Science, Carnegie Mellon University, USA, Email: mhilton@cmu.edu

***Abstract*—Prior research exploited the repetitiveness of code changes to enable several tasks such as code completion, bug-fix recommendation, library adaption, etc. These and other novel applications require accurate detection of semantic changes, but the state-of-the-art methods are limited to algorithms that detect specific kinds of changes at the syntactic level. Existing algorithms relying on syntactic similarity have lower accuracy, and cannot effectively detect semantic change patterns. We introduce a novel graph-based mining approach, CPATMINER, to detect *previously unknown* repetitive changes in the wild, by mining *fine-grained semantic code change patterns* from a large number of repositories. To overcome unique challenges such as detecting *meaningful* change patterns and scaling to large repositories, we rely on fine-grained change graphs to capture program dependencies.**

We evaluate CPATMINER by mining change patterns in a diverse corpus of 5,000+ open-source projects from GitHub across a population of 170,000+ developers. We use three complementary methods. First, we sent the mined patterns to 108 open-source developers. We found that 70% of respondents recognized those patterns as their meaningful frequent changes. Moreover, 79% of respondents even named the patterns, and 44% wanted future IDEs to automate such repetitive changes. We found that the mined change patterns belong to various development activities: adaptive (9%), perfective (20%), corrective (35%) and preventive (36%, including refactorings). Second, we compared our tool with the state-of-the-art, AST-based technique, and reported that it detects 2.1x more meaningful patterns. Third, we use CPATMINER to search for patterns in a corpus of 88 GitHub projects with longer histories consisting of 164M SLOCs. It constructed 322K fine-grained change graphs containing 3M nodes, and detected 17K instances of change patterns from which we provide unique insights on the practice of change patterns among individuals and teams. We found that a large percentage (75%) of the change patterns from individual developers are commonly shared with others, and this holds true for teams. Moreover, we found that the patterns are not intermittent but spread widely over time. Thus, we call for a community-based change pattern database to provide important resources in novel applications.

***Keywords*-Semantic Change Pattern Mining, Graph Mining**

I. INTRODUCTION

GRAND VALLEY STATE UNIVERSITY

GROUP B · KEY PAPER · THE 9 REVIEW QUESTIONS

Discovering Repetitive Code Changes in Python ML Systems R-CPatMiner

Dilhara, Ketkar, Sannidhi, Dig · ICSE 2022 · R-CPATMiner, the Python follow-up

Q1 · Main idea
Python ML developers repeat the same edits by hand. Find them, and find which ones need automation.

Q2 · Their research questions
Which change patterns are frequent in ML code? Does refactoring awareness help? How fast are the tools?

Q3 · Their solution
R-CPATMiner: CPatMiner for Python + type inference (PyType) + refactoring detection (PyRMiner).

Q4 · Gap they filled
CPatMiner worked only on Java and was confused by renames and moves.

Q5 · Method and results
28,308 patterns; **58%** appear in 2+ projects; 63% by 2+ authors; 71% of 97 developers want automation.

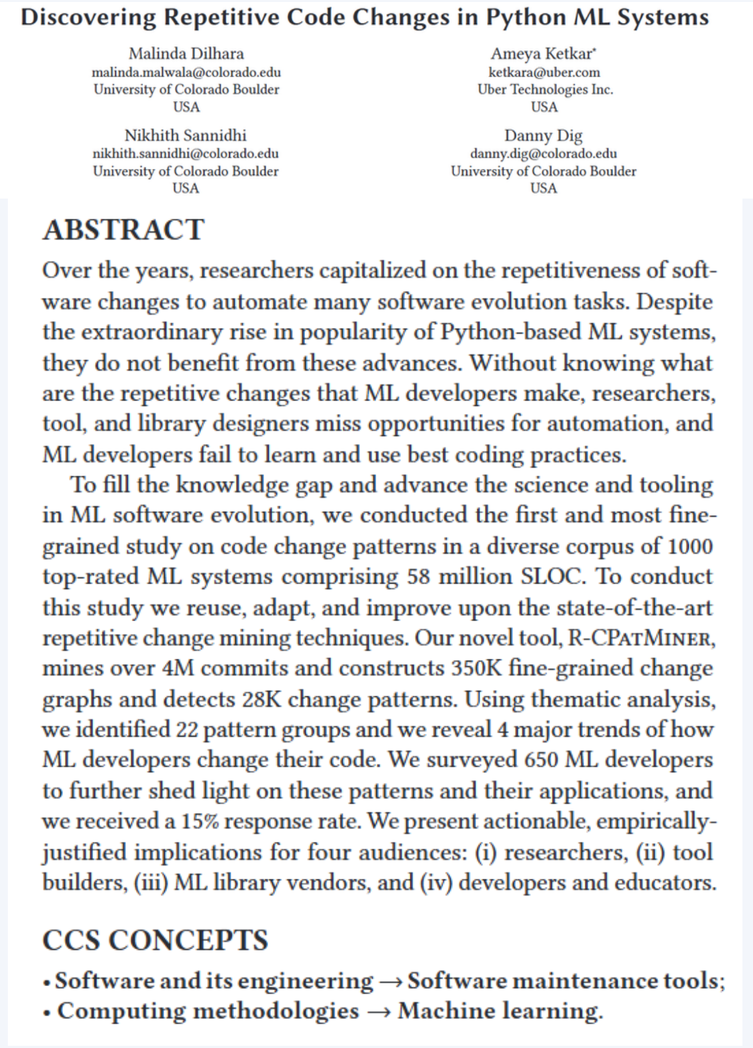
Q6 · Tools
R-CPATMiner, PyRMiner (≈95% precision), JavaFyPy, PyType.

Q7 · Data
1,000 ML repositories, 4.2 million commits. github.com/maldil/R-CPATMiner

Q8 · Future work / still open
A catalog of 22 pattern groups for tool builders to automate.

Q9 · Citation
dl.acm.org/doi/10.1145/3510003.3510225

Difference from my study: They mine all human edits in ML code. we mine only the human solutions of denied AI PRs. Their tools work on Python, my language.



Discovering Repetitive Code Changes in Python ML Systems

Malinda Dilhara
malinda.malwala@colorado.edu
University of Colorado Boulder
USA

Ameya Ketkar*
ketkara@uber.com
Uber Technologies Inc.
USA

Nikhith Sannidhi
nikhith.sannidhi@colorado.edu
University of Colorado Boulder
USA

Danny Dig
danny.dig@colorado.edu
University of Colorado Boulder
USA

ABSTRACT

Over the years, researchers capitalized on the repetitiveness of software changes to automate many software evolution tasks. Despite the extraordinary rise in popularity of Python-based ML systems, they do not benefit from these advances. Without knowing what are the repetitive changes that ML developers make, researchers, tool, and library designers miss opportunities for automation, and ML developers fail to learn and use best coding practices.

To fill the knowledge gap and advance the science and tooling in ML software evolution, we conducted the first and most fine-grained study on code change patterns in a diverse corpus of 1000 top-rated ML systems comprising 58 million SLOC. To conduct this study we reuse, adapt, and improve upon the state-of-the-art repetitive change mining techniques. Our novel tool, R-CPatMiner, mines over 4M commits and constructs 350K fine-grained change graphs and detects 28K change patterns. Using thematic analysis, we identified 22 pattern groups and we reveal 4 major trends of how ML developers change their code. We surveyed 650 ML developers to further shed light on these patterns and their applications, and we received a 15% response rate. We present actionable, empirically-justified implications for four audiences: (i) researchers, (ii) tool builders, (iii) ML library vendors, and (iv) developers and educators.

CCS CONCEPTS

• Software and its engineering → Software maintenance tools;
• Computing methodologies → Machine learning.

THE RESEARCH GAP

Two families, and the empty space between them

GROUP A

AI pull requests

✔️ AI mistakes
✔️ Denied PRs
✘ No rules
✘ No checker

THE EMPTY SPACE

Rules learned from the human solutions of denied AI pull requests

0

of n papers do both

GROUP B

Rules and checkers

✔️ Rules from fixes
✔️ Checkers
✔️ Repeat across projects
✘ Learned from human code

Our study sits here: Group A’s data (denied AI PRs + human solutions) with Group B’s method (rules and checkers).

GAP ANALYSIS · RESULT

What each family covers, and what is missing

	Group A	Group B	Our study
AI agent made the mistake	✓	✗	✓
Studies denied AI PRs	✓	✗	✓
Reads the human solution	◐	✓	✓
Turns it into a rule	✗	✓	✓
Tests repetition across projects	✗	✓	✓
Checks the next PR before merge	✗	✓	✓

Gap 1
The human solution is never read

Gap 2
No rules from AI mistakes

Gap 3
Repetition never measured for AI

Gap 4
No checker for new AI PRs

BACK TO THE STARTING QUESTIONS

What the literature answers, and what we will answer

FIRST QUESTION

What follow-up human action occurs after an AI pull request is denied?

THE LITERATURE SAYS

Often a human solves the same task in another PR (22.1% “resolved by another PR”), but nobody reads how the code was fixed.

OUR STUDY ANSWERS

We pair each denied AI PR with its human solution and compare the code.

SECOND QUESTION

What reusable obligation does the fix reveal, and can it catch the next mistake?

THE LITERATURE SAYS

Fixes can become accepted rules (73%) and changes repeat across projects (58–75%), but only shown for human code.

OUR STUDY ANSWERS

We write five-part obligations, test repetition (RQ2) and test a checker on new AI PRs (RQ3).

CONCLUSION · RECAP

Four things to remember

1

AI pull requests are often denied

and a human often solves the same task later.

2

Two families of papers

Group A has the AI data; Group B has the rule methods.

3

The empty space · GAP

0 of 48 papers learn rules from denied AI PRs.

4

Our study fills it

Denied AI PR → human solution → obligation → checker.

LITERATURE REVIEW AND RESEARCH GAP

THANK YOU

Evolution-Aware Assurance for Agent-Generated Software